

RunSpec interno como contrato único de execução em um Docker-like system construído com Python, Cython e C++

Pesquisa técnica aprofundada

12 de março de 2026

Sumário

1	Resumo executivo	3
2	O problema arquitetural que o RunSpec precisa resolver	5
3	Como o Docker original organiza esse problema	7
4	O RunSpec como contrato canônico do projeto	9
5	Taxonomia interna do RunSpec	11
6	Relação entre API externa, RunSpec e OCI	12
7	Definição aprofundada de cada conceito	13
7.1	Process spec	13
7.2	Environment variables	14
7.3	Current working directory (cwd)	14
7.4	Hostname	15
7.5	Mounts	15
7.6	Resources	16
7.7	Security	17
8	O contexto do projeto: Python no control plane, Cython na fronteira e C++ no data plane	19
9	Proposta de System Design	20
9.1	Fluxo sugerido de criação e início de um contêiner	21
10	Proposta de Low Level Design	22
10.1	Pipeline de compilação do RunSpec no control plane	24
10.2	Fronteira Cython	24
10.3	Núcleo nativo em C++	24
10.4	Estado operacional e supervisão	25
11	Domain-Driven Design aplicado ao RunSpec	26
12	Padrões de Design (GoF) relevantes	28
13	Requisitos funcionais sugeridos	30
14	Requisitos não funcionais sugeridos	32

15 A decisão mais importante de design: fazer do RunSpec um contrato de domínio, não um espelho do kernel	34
16 Plano de implementação incremental	35
17 Estratégia de testes e validação	37
18 Observações específicas sobre alguns campos críticos	38
18.1 Por que privileged não deveria ser conceito nativo do domínio	38
18.2 Por que rede deveria ficar fora do RunSpec v1	38
18.3 Por que o data plane precisa desconfiar do control plane	38
19 Conclusão	39
20 Referências	40

Capítulo 1

Resumo executivo

A feature proposta, denominada aqui de *RunSpec interno*, deve ser entendida como o contrato canônico e imutável que descreve tudo aquilo que o seu sistema precisa saber para transformar uma intenção de execução em um processo isolado pelo kernel Linux. Em termos práticos, isso significa que o RunSpec não é apenas um objeto de API, nem apenas um arquivo de configuração, nem apenas um DTO que cruza a fronteira entre Python e C++. Ele é o eixo semântico que unifica o que o usuário pediu, o que a política da plataforma permitiu, o que a imagem declarou como padrão, o que o sistema resolveu em termos de paths, usuários e perfis de segurança, e o que o *data plane* precisa aplicar de modo determinístico no momento da criação do contêiner.

No Docker histórico, esse papel nunca aparece como um único objeto puro. O modelo foi se formando ao longo do tempo por meio da combinação de Config, HostConfig, NetworkingConfig, estado persistido no daemon, resolução do WorkingDir e do Entrypoint, materialização de *root filesystem*, geração de *bundle* OCI e invocação de *runtime* compatível com OCI por intermédio do containerd e do runc [1][2][3][4][5]. Já no ecossistema OCI, existe um contrato de execução muito mais nítido, expresso pelo `config.json` do *runtime bundle*, com objetos para process, mounts, hostname, `linux.resources`, `linux.namespaces`, `capabilities`, `seccomp`, `maskedPaths`, `readonlyPaths` e outros campos necessários para descrever o ambiente efetivo de execução [6][7][8].

A principal tese desta pesquisa é que, para um projeto novo e deliberadamente desenhado em camadas, vale a pena fazer aquilo que o Docker original não pôde fazer por razões históricas: adotar um RunSpec interno único, versionado, imutável após admissão, semanticamente mais limpo que o modelo externo de API e ainda assim traduzível para OCI na borda inferior do sistema. O plano mais sólido é manter, em Python, toda a camada de *control plane*, metadados, orquestração, API, políticas e compilação de especificações; concentrar, em C++, a parte que fala com o kernel, controla processos, namespaces, *mounts*, *cgroups*, segurança e supervisão de vida do contêiner; e utilizar Cython como fronteira tipada e de baixa sobrecarga entre os dois lados, liberando o GIL nas operações demoradas e evitando que objetos Python escapem para o caminho quente do *runtime* [9][10][11][12][25][26].

Essa decisão resolve vários problemas de engenharia ao mesmo tempo. Ela evita que detalhes históricos de API vazem para o núcleo do domínio, impõe determinismo ao processo de admissão e persistência da intenção de execução, permite auditar com precisão o que foi solicitado e o que foi efetivamente aplicado, e cria um ponto de acoplamento estável entre um *control plane* de evolução rápida, escrito em Python, e um *data plane* nativo, escrito em C++, que precisa ser estrito, defensivo e previsível. Ela também ajuda a separar com clareza três coisas que tendem a se confundir em plataformas de contêineres: a intenção

declarativa, o contrato admitido de execução e o plano concreto de aplicação no kernel.

A pesquisa conclui que o RunSpec deve ser modelado como um objeto canônico de domínio, subdividido em pelo menos seis áreas semânticas: `process`, `env`, `cwd`, `hostname`, `mounts`, `resources` e `security`, sendo `env` e `cwd` logicamente subordinados ao `process`, `hostname` subordinado ao contexto UTS, `mounts` subordinado à montagem de *rootfs* e sistema de arquivos, `resources` subordinado ao domínio de `cgroups` e controle de consumo, e `security` subordinado ao envelope de isolamento e mitigação. O modelo deve ser Linux-first, com `cgroup v2` como alvo principal, OCI como contrato de interoperabilidade, e uma política de segurança cuja postura padrão seja a de menor privilégio. Também deve tratar `cwd` como campo de alto risco, não como simples detalhe cosmético, porque vulnerabilidades recentes do `runc` mostraram que esse atributo pode ser vetor de escape quando mal validado [23].

No plano de implementação, a recomendação é organizar o sistema em três níveis. O primeiro nível é o da intenção externa, exposta por API, CLI ou futuramente por compatibilidade Docker-like. O segundo é o RunSpec interno, produzido por um compilador de especificações em Python que combina imagem, *defaults* da plataforma, políticas de segurança, limites de recursos e regras de resolução. O terceiro é o plano de execução nativo, em C++, que pode ser representado internamente e também materializado como *bundle* OCI para inspeção, interoperabilidade e testes. Essa estratégia permite que o seu sistema seja, ao mesmo tempo, semanticamente coerente para o domínio e pragmático em relação ao ecossistema real.

Capítulo 2

O problema arquitetural que o RunSpec precisa resolver

Todo sistema de contêineres precisa responder à mesma pergunta fundamental: qual é o contrato exato entre a intenção do usuário e a criação do processo isolado? Em projetos pequenos, essa pergunta costuma ser escondida por trás de chamadas ad hoc, flags de linha de comando e parâmetros espalhados por funções. Em projetos maduros, ela reaparece como um problema central de arquitetura porque o sistema passa a ter de explicar, persistir, versionar, auditar, reconciliar e reproduzir decisões de execução. Nesse ponto, um contrato interno bem desenhado deixa de ser detalhe de implementação e passa a ser a espinha dorsal da plataforma.

A feature descrita pelo usuário é particularmente relevante porque toca exatamente esse ponto. Ao declarar “RunSpec interno — contrato único de execução; process spec, env, cwd, hostname, mounts, resources, security”, a proposta reconhece que o objeto principal do sistema não deve ser “container” como substantivo genérico, mas sim “especificação de execução admitida”. O contêiner é o resultado operacional; o RunSpec é a descrição daquilo que precisa acontecer para que ele exista de forma reproduzível. Essa inversão é valiosa porque desloca o foco do artefato operacional para o contrato de domínio.

Em uma arquitetura dividida entre *control plane* em Python e *data plane* em C++, a ausência desse contrato tende a criar acoplamentos ruins. O lado Python começa a trafegar objetos semânticos demais, carregados de decisões ainda não resolvidas, e o lado C++ acaba recebendo estruturas parcialmente interpretadas, tendo de adivinhar precedências, recomputar *defaults* ou aceitar ambiguidades de segurança. O resultado típico é o surgimento de um “proto-domínio” em cada lado da fronteira, com semânticas parecidas mas não idênticas. Em pouco tempo, a plataforma passa a ter duas verdades sobre o que significa rodar um contêiner. O RunSpec existe precisamente para impedir esse tipo de deriva.

Esse contrato também é necessário para atacar um segundo problema: o descompasso entre o que o usuário pede e o que o sistema realmente aplica. No mundo real, quase nunca se executa exatamente o que veio na requisição original. Há herança da imagem, há *defaults* do daemon, há políticas organizacionais, há resolução de caminhos, há inferência de perfis de segurança, há injeção de variáveis, há transformação para o formato do *runtime*. Se o sistema não materializa um contrato intermediário e canônico, fica muito difícil responder perguntas operacionais e forenses elementares, como “o que exatamente foi admitido?”, “quais *mounts* ficaram efetivos?”, “qual perfil seccomp foi aplicado?”, “qual *cwd* foi realmente usado?” e “por que esta execução diverge daquela outra?”.

O RunSpec interno deve, portanto, resolver simultaneamente problemas de semântica, persistência, evolução e segurança. Ele precisa ser rico o suficiente para representar as intenções necessárias ao *data plane*, mas restrito o suficiente para impedir ambiguidades e combinações inválidas. Precisa ser estável o bastante para ser persistido e auditado, mas versionável para acomodar evolução do sistema e do kernel. Precisa ser legível do ponto de vista humano, mas orientado a dados e determinístico do ponto de vista de máquina. E, acima de tudo, precisa ser tratado como um contrato que o *data plane* pode consumir sem “adivinhar” nada.

Capítulo 3

Como o Docker original organiza esse problema

O Docker original é uma referência inevitável, mas é importante interpretá-lo com cuidado. Seu modelo não nasceu como um sistema limpo, desenhado do zero ao redor de uma única abstração semântica. Ele foi evoluindo para atender UX, compatibilidade histórica, ecossistema de imagens, necessidades do daemon e, mais tarde, a separação de responsabilidades entre Docker Engine, containerd e runc. Por isso, o seu desenho contém excelentes soluções práticas, mas também cicatrizes de evolução.

Na documentação oficial do Docker Engine, o sistema é apresentado como uma arquitetura cliente-servidor em que o daemon `dockerd` expõe uma API, recebe comandos do cliente e gerencia imagens, contêineres, redes e volumes [1]. Essa documentação também deixa explícito que o Engine atual utiliza o `containerd` para o ciclo de vida de contêineres e que, por padrão, o `containerd` usa `runc` como *runtime* de baixo nível [2]. Outra parte da documentação ressalta que o daemon depende de um *runtime* compatível com OCI por meio do `containerd` e que este se encarrega de interagir com namespaces, cgroups e mecanismos de segurança do Linux [2]. Em outras palavras, a fronteira superior do Docker fala a linguagem de API e objetos do daemon; a fronteira inferior fala a linguagem de OCI e kernel.

No código do Moby, isso aparece de forma muito clara. O tipo `Config` concentra informações consideradas portáteis, isto é, atributos que fazem sentido mesmo fora de um host específico, como `Hostname`, `User`, `Env`, `Cmd`, `Image`, `Volumes`, `WorkingDir`, `Entrypoint`, `Labels` e outros campos ligados à identidade do contêiner e ao processo principal [3]. O tipo `HostConfig`, por sua vez, foi explicitamente definido para abrigar informações não portáteis, dependentes do host, como *bind mounts*, modo de rede, capacidades, DNS, *extra hosts*, modos de PID e UTS, opção `Privileged`, `ReadonlyRootfs`, `SecurityOpt`, `Tmpfs`, `Sysctls`, `Runtime`, `Resources`, lista de *mounts* e caminhos mascarados ou somente leitura [4]. A API HTTP de criação de contêineres reproduz a mesma ideia ao exigir um corpo JSON em que parte dos campos fica no corpo principal e parte dentro do objeto `HostConfig` [5].

Esse arranjo funciona, mas ele não deve ser tomado como modelo interno ideal para um projeto novo. Ele é, ao mesmo tempo, contrato de API, mecanismo de compatibilidade, formato de persistência histórica do daemon e insumo para uma cadeia de transformação posterior. Para quem está construindo um sistema do zero, esse desenho traz mais ruído do que benefício se for copiado literalmente. O ganho real está em entender o que o Docker está separando semântica e operacionalmente, e não em repetir a exata forma de seus objetos.

Quando descemos um nível, o papel do OCI fica ainda mais importante. O OCI Runtime Specification define um *bundle* com um `config.json` que descreve o ambiente de execução do contêiner, incluindo `process`, `root`, `mounts`, `hostname`, `linux.resources`, `linux.namespaces`, `capabilities`, `seccomp`, `maskedPaths`, `readonlyPaths`, `mountLabel`, `apparmorProfile`, `selinuxLabel` e outros elementos [6][7][8]. O `runc` consome precisamente esse modelo ao exigir um *bundle* com `config.json` e `rootfs` [13]. O `containerd`, por sua vez, trabalha com a ideia de tarefa e processo, podendo receber um Spec OCI completo para a criação da tarefa inicial e um `process spec` para operações de `exec` subsequentes [10][11].

A leitura correta desse ecossistema é a seguinte. No Docker, não existe um único objeto elegante que vá do topo ao kernel. Existem camadas sucessivas de descrição, resolução e adaptação. O seu projeto tem a oportunidade de tornar isso explícito e limpo. Em vez de carregar para dentro do domínio a divisão histórica entre `Config` e `HostConfig`, o ideal é usá-la apenas como evidência de que existem dimensões diferentes de configuração. Em vez de expor diretamente OCI como contrato de negócio, o ideal é usá-lo como alvo de tradução e como *ground truth* para as propriedades necessárias à execução em Linux. O `RunSpec` interno deve ficar entre esses dois mundos: semanticamente mais claro do que a API do Docker e mais próximo do domínio do produto do que o `config.json` do OCI.

Capítulo 4

O RunSpec como contrato canônico do projeto

No contexto do projeto proposto, o RunSpec deve ser definido como a representação imutável, persistível e versionada da execução admitida de um contêiner. A palavra “admitida” é crucial. Ela significa que o objeto não é apenas o pedido bruto do usuário, nem apenas o reflexo da imagem, nem apenas a configuração final do *runtime* tal como escrita em disco. Ele é o resultado de um processo de compilação semântica realizado pelo *control plane*: validação sintática, validação semântica, aplicação de políticas, resolução de precedências, enriquecimento por padrões da imagem, normalização de caminhos e limites, congelamento de valores e atribuição de identidade.

Essa definição resolve um erro comum em projetos de infraestrutura: usar o mesmo objeto para intenção, admissão e execução. Essas três coisas parecem próximas, mas não são iguais. A intenção externa pode conter alias, omissões, atalhos de UX, campos redundantes e combinações permissivas. A admissão interna já precisa ser explícita, completa e determinística. A execução, por sua vez, pode exigir ainda mais materialização, como caminhos absolutos resolvidos, perfis de segurança compilados, tradução para OCI e detalhes que são específicos do host ou do kernel. Quando o mesmo objeto tenta cumprir os três papéis, o sistema fica semanticamente instável.

Uma boa forma de estruturar esse espaço é reconhecer três níveis formais de contrato. O primeiro é o `ContainerIntent` ou equivalente, que representa o que vem da API pública. O segundo é o RunSpec, que representa o contrato canônico admitido. O terceiro é o `ExecutionPlan`, ou `RuntimePlan`, que representa a forma nativa e concreta pela qual o *data plane* aplicará o contrato, podendo inclusive ser materializado como *bundle* OCI. O usuário pediu especificamente o RunSpec, mas ele se fortalece quando posicionado como o nível intermediário e central entre esses dois extremos.

Na prática, isso quer dizer que o RunSpec não deve carregar ambiguidades de UX. Ele não deve, por exemplo, depender de regras implícitas espalhadas entre imagem, CLI e daemon para descobrir qual é o comando final. O campo de processo já deve conter um `argv` ou uma combinação normalizada de `entrypoint` e `args` cujo resultado final seja inequívoco. Da mesma forma, o `cwd` já deve ter sido derivado da imagem ou definido explicitamente; o `hostname` já deve ter sido validado contra a política de namespace UTS; os *mounts* já devem ter sido ordenados e validados; os recursos já devem estar em um formato coerente com `cgroup v2`; e a segurança já deve estar consolidada em um envelope claro de namespaces, capacidades, `no_new_privs`, perfis LSM e `seccomp`.

O contrato também precisa ser imutável após admissão. Isso não significa que o sistema não possa suportar operações como `exec`, `resize` de terminal, `kill` ou atualização de certos recursos. Significa apenas que essas ações não devem mutar silenciosamente o RunSpec original. Para preservar rastreabilidade e coerência de domínio, o sistema deve tratar o RunSpec inicial como um instantâneo canônico da criação do contêiner. Operações posteriores podem gerar comandos específicos, como ExecSpec para novos processos ou ResourcePatch para limites mutáveis. Esse desenho reflete a própria separação observada no containerd, onde a criação da tarefa usa um Spec completo e execuções posteriores usam um process spec próprio [10][11].

Capítulo 5

Taxonomia interna do RunSpec

Do ponto de vista conceitual, o RunSpec deve ser organizado em subestruturas semanticamente coesas. A primeira é `process`, que descreve o processo inicial e inclui o comando efetivo, o ambiente, o diretório de trabalho, o usuário, a necessidade de terminal, limites de recursos associados ao processo e afins. A segunda é `hostname`, que representa a identidade UTS do contêiner. A terceira é `mounts`, que descreve a topologia adicional de sistema de arquivos além do `rootfs`. A quarta é `resources`, que governa CPU, memória, PIDs, I/O e dispositivos. A quinta é `security`, que consolida namespaces, capacidades, `no_new_privs`, `seccomp`, AppArmor, SELinux, mascaramento de caminhos, caminhos somente leitura, modo privilegiado se for suportado por compatibilidade e quaisquer políticas correlatas. Essas categorias dialogam diretamente com OCI [6][7] e, ao mesmo tempo, se alinham com o que o Moby separa entre `Config`, `HostConfig` e `Resources` [3][4].

Ainda assim, é recomendável que o RunSpec tenha campos de metadados próprios, como `version`, `id`, `name`, `image_ref`, `rootfs_ref`, `created_at`, `annotations`, `labels` e talvez `profiles` ou `policy_refs`. Esses campos não são o foco desta feature, mas são essenciais para que o contrato exista como artefato durável do domínio. Uma plataforma de contêineres não lida apenas com parâmetros de execução; ela lida com entidades persistidas, reconciliadas e observadas ao longo do tempo.

Uma decisão particularmente importante é a representação de `env`. Docker e OCI o representam como lista ordenada de strings `KEY=VALUE` [5][6]. Isso preserva fidelidade com a forma como o ambiente é passado ao processo no sistema operacional, mas também carrega ambiguidades em relação a duplicidade de chaves. Um projeto novo pode se sentir tentado a modelar ambiente como dicionário simples, porém isso perde a fidelidade total e pode introduzir diferenças sutis de precedência. A solução mais segura é manter internamente uma sequência ordenada de pares chave-valor, com regra clara de deduplicação durante a compilação do RunSpec. Assim, o sistema mantém rastreabilidade e compatibilidade com OCI, sem abrir mão de semântica controlada.

Outra decisão central é a modelagem de `mounts`. No OCI, a ordem dos `mounts` importa e os destinos devem ser caminhos absolutos dentro do contêiner [6]. Isso é crítico porque a sobreposição de montagens altera a visibilidade do sistema de arquivos. A documentação do Docker também alerta que `bind mounts` podem obscurecer conteúdo existente e são graváveis por padrão, o que cria implicações fortes de segurança e portabilidade [22]. O RunSpec deve preservar explicitamente a ordem, o tipo de `mount`, o destino, o modo de acesso, as opções de propagação e os dados necessários para resolver a origem. Ele não deve tratar `mounts` como um conjunto indiferenciado.

Capítulo 6

Relação entre API externa, RunSpec e OCI

Uma das decisões mais valiosas para o projeto é não colapsar o contrato público no contrato interno, nem o contrato interno no contrato OCI. O Docker original sofreu com essa sobreposição parcial porque sua API pública acumulou responsabilidade histórica demais. Em um projeto novo, o RunSpec deve ser deliberadamente interno. A API pública pode continuar parecida com a semântica de mercado, inclusive para facilitar adoção, mas ela deve ser traduzida por um compilador de especificações que pertença ao *control plane*. Esse compilador é o lugar onde se combinam imagem, *defaults* da plataforma, políticas e validações.

Do outro lado, o *data plane* não deve depender semanticamente do formato da API pública. Ele deve depender do RunSpec ou de um plano nativo derivado dele. E, sempre que possível, esse plano derivado deve ser traduzível para OCI. Essa recomendação tem quatro vantagens. A primeira é a possibilidade de reutilizar o ecossistema de testes, conceitos e ferramentas de OCI. A segunda é a legibilidade operacional, já que o *bundle* OCI materializado é um artefato bem conhecido. A terceira é a redução do acoplamento entre seu domínio interno e a implementação do *runtime*. A quarta é a possibilidade futura de interoperação com *runc*, *crun* ou outro *runtime* compatível, caso isso faça sentido.

A melhor forma de pensar nessa arquitetura é como uma ampulheta. Na parte superior, existem múltiplas formas de entrada: API HTTP, CLI, talvez importação de formatos compatíveis. Na cintura da ampulheta, existe um único contrato interno forte: o RunSpec. Na parte inferior, existem um ou mais alvos concretos: OCI bundle, chamadas nativas ao *runtime*, supervisores, shims e syscalls. Quanto mais forte for essa cintura, menos o sistema sofrerá com deriva semântica e mais fácil será evoluir suas bordas.

Capítulo 7

Definição aprofundada de cada conceito

7.1 Process spec

No OCI, o objeto `process` descreve o processo a ser iniciado dentro do ambiente do contêiner, incluindo terminal, tamanho de console, diretório de trabalho, ambiente, argumentos, capacidades e atributos de segurança [6][7]. No Docker, a mesma preocupação aparece distribuída entre `Entrypoint`, `Cmd`, `User`, `Env`, `WorkingDir`, `Tty`, `OpenStdin` e campos correlatos da configuração do contêiner [3][5]. Em ambos os casos, o problema real não é apenas “qual comando rodar”, mas “como representar de forma inequívoca a identidade operacional do processo inicial”.

Para o projeto proposto, `process spec` deve ser tratado como um subdomínio próprio dentro do `RunSpec`. Ele precisa capturar a linha de execução efetiva, e não apenas o comando declarado pelo usuário. Isso implica resolver a precedência entre o que vem da imagem e o que vem da requisição. Em imagens Docker, `Entrypoint` e `Cmd` se combinam de modo específico; se a API pública do seu sistema pretender espelhar essa UX, essa composição deve acontecer antes da materialização do `RunSpec`. O objeto final já deve conter um `argv` completo, ordenado e pronto para ser traduzido a `process.args` do OCI.

Além do comando, o `process spec` deve modelar o usuário efetivo. Em Docker, o campo `User` é frequentemente passado como string e depois resolvido no ambiente da imagem. Em um sistema novo, vale separar a intenção simbólica da resolução efetiva. O `RunSpec` pode carregar tanto a representação declarada quanto a resolução final em UID, GID e grupos suplementares, se disponíveis no momento de admissão. Caso essa resolução dependa do `rootfs`, há duas alternativas legítimas: resolvê-la apenas no *data plane* mantendo no `RunSpec` a intenção declarada, ou introduzir uma etapa explícita de resolução durante a preparação do `rootfs`. O importante é que a semântica seja clara e auditável.

Também é recomendável incluir no `process spec` atributos que, no mercado, às vezes ficam dispersos, como TTY, política de *stdio*, tamanho de terminal, sinais, *rlimits* por processo e talvez *oom_score_adj*. Mesmo que uma primeira versão não suporte todos eles, o desenho do objeto deve admitir essa evolução. O erro de muitas implementações é reduzir `process` a `argv`; com isso, campos como `cwd`, `env`, usuário, terminal e privilégio acabam vazando para outras subestruturas sem um centro de gravidade. No seu caso, `process` deve ser o lugar onde essas dimensões se ancoram semanticamente.

No nível de LLD, o `process spec` precisa ser convertido para uma estrutura nativa em C++ sem carregamento de ambiguidade. Isso sugere que o Cython faça a ponte por meio de tipos simples e *ownership* explícito, por exemplo vetores de strings, estruturas triviais para usuário e *flags* booleanas para TTY, evitando callbacks Python no caminho crítico. Se o sistema optar por serializar o `RunSpec` entre processos, é

ainda melhor transportar um esquema estável e já normalizado, em vez de converter múltiplas vezes estruturas dinâmicas de Python.

7.2 Environment variables

No Docker, variáveis de ambiente aparecem tanto na imagem quanto na configuração de execução, e a API as transmite como lista de strings. No OCI, `process.env` adota o mesmo formato [5][6]. Isso é importante porque o ambiente do processo, em nível de sistema operacional, é de fato um vetor de strings passado ao `execve`, e não um mapa abstrato. Contudo, um sistema de plataforma precisa impor semântica adicional. Ele precisa saber de onde veio cada variável, qual prevalece em caso de conflito, quais chaves são reservadas pela plataforma, quais são injetadas por segurança ou observabilidade e quais não podem ser sobrepostas.

Por essa razão, o RunSpec deveria tratar `env` como resultado de uma compilação de camadas. Uma ordem razoável de precedência é: ambiente definido pela imagem, depois *defaults* da plataforma, depois variáveis exigidas por políticas, depois *overrides* do usuário, e por fim variáveis reservadas cuja escrita é monopolizada pelo sistema. O ponto crucial é que essa ordem precisa ser documentada e testada. O pior cenário é um sistema em que o usuário não sabe se `PATH`, `HOSTNAME` ou `LANG` virão da imagem, da plataforma ou do `daemon`, porque isso causa comportamento irreproduzível.

Outro detalhe importante é a serialização. Se o RunSpec quiser fidelidade total com OCI, pode armazenar `env` como vetor ordenado de pares ou mesmo como vetor de strings já formatadas. Se quiser melhor ergonomia no domínio, pode armazenar uma estrutura mais rica durante a etapa de compilação e produzir o vetor final apenas ao congelar o contrato. O que não convém é trafegar um dicionário solto e, depois, confiar em detalhes de implementação de ordenação ou deduplicação. O ambiente precisa ser determinístico, especialmente porque sua alteração muda o hash efetivo da execução.

Do ponto de vista de segurança, `env` não é neutro. Variáveis como `LD_PRELOAD`, `LD_LIBRARY_PATH`, `PATH`, `PYTHONPATH`, `SSL_CERT_FILE`, `HOME`, `TMPDIR` e equivalentes podem alterar radicalmente o comportamento do processo. Um RunSpec sério deve permitir políticas que bloqueiem ou limitem certas chaves em contextos sensíveis. Também deve considerar a proteção de segredos. Em muitos sistemas, variáveis de ambiente são usadas para segredos por conveniência, mas isso facilita vazamento para *logs*, *debug dumps* e inspeção de processos. Um desenho maduro pode aceitar `env` no RunSpec, mas modelar segredos como referências a outro subsistema, injetando valores apenas no plano final ou por *mounts* temporários de arquivos.

7.3 Current working directory (cwd)

O `cwd` parece um detalhe inocente até que ele quebra isolamento. No OCI, `process.cwd` é obrigatório e deve ser caminho absoluto [6]. No Docker, o `WorkingDir` aparece na configuração do contêiner e também pode ser herdado da imagem [3][5]. À primeira vista, isso sugere um simples problema de conveniência: em qual diretório o processo inicial começará? Na realidade, `cwd` é um componente estrutural do modelo de segurança do contêiner.

Essa importância ficou ainda mais clara com a vulnerabilidade corrigida no `runc` em 2024, na qual combinações de `process.cwd` e descritores de arquivo internos permitiam que um processo iniciasse com diretório fora do ambiente esperado, inclusive via caminhos sob `/proc/self/fd/...`, abrindo caminho para *breakout* [23]. O aviso oficial deixa claro que o problema exigiu endurecimento no fechamento de FDs internos, validação do diretório final e rejeição de casos em que o *working directory* efetivo não estivesse de fato dentro do sistema de arquivos do contêiner [23]. A lição arquitetural é simples e não negociável: `cwd` precisa

ser tratado como campo de segurança.

No seu projeto, isso implica várias medidas. Em primeiro lugar, o RunSpec deve exigir `cwd` absoluto após a fase de compilação. Se a intenção externa vier vazia, a ausência deve ser resolvida a partir da imagem ou cair para `/`. Em segundo lugar, o *control plane* deve validar o formato, mas o *data plane* deve revalidar o valor no contexto da montagem final. Em terceiro lugar, a validação não pode ser meramente textual; ela deve considerar resolução de links, montagem efetiva de `rootfs`, `pivot_root` ou `chroot`, e o fato de que descritores de arquivo ou links mágicos em `/proc` podem subverter a expectativa [23][24].

Uma política robusta inclui pelo menos as seguintes garantias conceituais: o caminho deve ser absoluto, a resolução final deve permanecer dentro do `rootfs` do contêiner, o diretório deve existir ou ser criado segundo regra explícita da plataforma, caminhos sob `/proc/self/fd` e equivalentes devem ser proibidos, e o valor final deve ser validado novamente depois que o ambiente de *mounts* e *root* estiver estabilizado. Além disso, é prudente persistir tanto o `cwd` solicitado quanto o `cwd` efetivamente aplicado, caso a política permita normalizações. Em matéria de isolamento, auditar o valor real é tão importante quanto auditar o valor pedido.

7.4 Hostname

O `hostname` é mais simples que `cwd`, mas continua relevante. No OCI, `hostname` é um campo de topo e sua aplicação está ligada à existência de um namespace UTS adequado [6][7]. No Docker, ele aparece como parte da configuração portátil do contêiner [3][5]. Conceitualmente, o `hostname` atua como identidade local do contêiner para o processo e para aplicações que consultam esse nome no ambiente interno.

A principal questão de projeto aqui é o acoplamento com a política de namespaces. Se o contêiner roda em seu próprio namespace UTS, definir `hostname` é operação natural. Se ele compartilha UTS com o host ou com outro contêiner, a operação pode ser inválida, perigosa ou semanticamente enganosa. Portanto, o RunSpec não deveria tratar `hostname` como simples string isolada; ele precisa ser coerente com a subestrutura de segurança e namespaces. Uma combinação de “hostname customizado” com “UTS compartilhado” deveria ser rejeitada pelo compilador do RunSpec ou, no mínimo, pela validação de admissão.

Há ainda a questão de precedência. Docker frequentemente injeta `HOSTNAME` no ambiente interno de forma derivada do nome da instância ou do `hostname` configurado. Em um sistema novo, isso precisa ser explícito. O ideal é que `hostname` seja atributo próprio, que possa eventualmente gerar uma variável de ambiente reservada, mas sem depender de convenções implícitas e pouco auditáveis. Em outras palavras, `hostname` não deve viver escondido dentro de `env`; ele deve continuar sendo um elemento primeiro do contrato.

Do ponto de vista de DDD, `hostname` também ajuda a separar identidade operacional de identidade de metadados. O nome do contêiner usado pela API, o identificador persistido e o `hostname` visto pelo processo podem coincidir ou não. Essas três identidades cumprem papéis diferentes. Misturá-las no mesmo campo costuma gerar bugs de UX e de depuração.

7.5 Mounts

Poucos campos têm tanto impacto operacional quanto `mounts`. No OCI, `mounts` é o conjunto de montagens adicionais aplicadas ao ambiente do contêiner, além do `rootfs`, e cada entrada possui destino, tipo, origem e opções [6]. A especificação também ressalta que a ordem importa e que os destinos devem ser caminhos absolutos dentro do contêiner [6]. No Docker, *bind mounts*, volumes e *tmpfs* aparecem em múltiplas formas

de configuração, inclusive como *binds* tradicionais e como lista estruturada de *mounts* em `HostConfig` [4][5][22].

No seu projeto, *mounts* merecem um subdomínio próprio porque concentram semântica de portabilidade, segurança e resolução de recursos. Um *bind mount* para um caminho do host é algo completamente diferente de um volume gerenciado, de um `tmpfs`, de um dispositivo, de uma montagem de segredo ou de uma montagem produzida por `snapshotter`. Todas essas coisas podem ser serializadas em algo parecido no fim, mas pertencem a classes distintas de origem e governança. O `RunSpec` deve refletir essa diferença.

A melhor prática é modelar *mounts* em dois níveis. O primeiro é o nível declarativo, ainda orientado ao domínio, em que a origem pode ser um `source_ref` abstrato, um tipo lógico de *mount* e intenções como somente leitura, propagação e opções específicas. O segundo é o plano de montagem resolvido, em que cada entrada já contém caminho efetivo, parâmetros concretos e verificações aplicadas. Esse segundo nível pode viver no `ExecutionPlan`, não necessariamente no `RunSpec`. A distinção é útil porque um caminho do host resolvido cedo demais faz o contrato perder portabilidade e pode vaziar detalhes do nó de execução para a camada errada.

A documentação do Docker sobre *bind mounts* é clara ao lembrar que eles são graváveis por padrão, obscurecem conteúdo pré-existente e acoplam o contêiner à estrutura do host [22]. Isso deveria se refletir na postura padrão do seu sistema. Uma plataforma nova, preocupada com segurança e reprodutibilidade, deve tornar *bind mounts* explícitos e conservadores. O padrão recomendável é leitura apenas, com escrita exigindo manifestação clara. Também convém normalizar propagação para `rprivate` ou equivalente seguro, salvo necessidade explícita em contrário. *Mounts* sobre caminhos sensíveis, como `/proc`, `/sys`, diretórios de runtime e locais usados por o próprio supervisor, precisam ser bloqueados por política.

Outro aspecto decisivo é a interação entre *mounts* e `cwd`. O diretório de trabalho só faz sentido à luz da topologia final de sistema de arquivos. Se um `cwd` é validado antes, mas um *mount* posterior encobre o caminho ou o redireciona para local inesperado, a validação inicial perde valor. Isso é mais um motivo para que o *data plane* revalide invariantes depois da composição final de *rootfs* e *mounts*. Em termos de LLD, o motor de montagem e o validador de `cwd` deveriam compartilhar a mesma visão de namespace de montagem.

7.6 Resources

No ecossistema OCI, o bloco `linux.resources` mapeia o contrato de consumo para as primitivas de `cgroups`, abrangendo CPU, memória, bloco de I/O, dispositivos, limites de *huge pages*, limite de PIDs e extensões específicas como `unified` para `cgroup v2` [7]. No código do Moby, a estrutura `Resources` mostra um acúmulo histórico de parâmetros ligados a memória, CPU, `blkio`, `cpuset`, dispositivos e afins [4]. Já a documentação do kernel Linux para `cgroup v2` deixa claro que a interface moderna e preferencial é a do modelo unificado, com arquivos de controle voltados para composição hierárquica e governança consistente de recursos [15].

Para um projeto novo, a recomendação é objetiva: o `RunSpec` deve nascer com `cgroup v2` como modelo principal, mesmo que o sistema mantenha algum grau de compatibilidade com representações legadas. Isso não significa ignorar o vocabulário popular de Docker, como `memory`, `nano_cpus` ou `cpuset_cpus`, mas sim traduzi-lo cedo para uma semântica interna coerente com `cgroup v2`. O custo de carregar, no centro do domínio, as ambiguidades de `v1` e `v2` é desnecessário e alto.

Também é importante separar aquilo que é estruturalmente parte do `RunSpec` daquilo que pode ser alterado

em *runtime*. Alguns limites de recursos são definidos na criação e raramente mudam; outros, como certas quotas ou pesos, podem ser atualizados. Em vez de permitir mutação in-place do RunSpec, o sistema deve descrever explicitamente quais controles aceitam reendo operacional posterior, por meio de um `ResourcePatch` ou comando de atualização. Essa separação preserva a imutabilidade do contrato inicial sem impedir operações administrativas.

O bloco `resources` do RunSpec precisa ainda conversar com o domínio de governança. Uma plataforma não controla só limites absolutos; ela também expressa garantias, classes de serviço, prioridades e herança a partir de perfis. Isso sugere que o domínio de recursos tenha seus próprios valores e perfis de política, compilados para o formato final apenas na fase de admissão. Em DDD, isso justifica tratá-lo como *bounded context* relevante, não apenas como saco de números.

7.7 Security

A subestrutura `security` é, de longe, a mais ampla e crítica do RunSpec. O OCI trata segurança de forma distribuída entre `linux.namespaces`, `capabilities`, `seccomp`, `apparmorProfile`, `selinuxLabel`, `maskedPaths`, `readonlyPaths`, `mountLabel`, além de controles ligados a `noNewPrivileges` e *mappings* de usuário quando há namespace de usuário [6][7]. O kernel Linux fornece as bases conceituais: namespaces para isolamento de visão do sistema [14], cgroups para governança de recursos [15], capacidades para decompor privilégios de root [16], `no_new_privs` para impedir ganho de privilégio em `execve` [18] e `seccomp` filtrado por BPF para controle de syscalls [17]. O Docker, por sua vez, combina esses mecanismos com perfis padrão de `seccomp`, AppArmor, opções de `SecurityOpt`, suporte a rootless e `userns-remap` [19][20][21].

Em um projeto novo, o primeiro princípio deveria ser não representar segurança como coleção amorfa de “flags especiais”. Segurança é um envelope coerente. Isso quer dizer que o RunSpec precisa agrupar, de forma explícita, as decisões sobre isolamento de namespaces, identidade de usuário, capacidade de ganhar privilégios, perfil `seccomp`, perfil LSM, política de sistema de arquivos, dispositivos e eventuais `sysctls` permitidos. O segundo princípio é ainda mais importante: o *data plane* deve tratar o conteúdo do bloco `security` como insumo não confiável, mesmo que ele venha do próprio *control plane*. A fronteira com o kernel é um lugar onde a revalidação é obrigatória.

As capacidades Linux merecem atenção especial. O `man 7 capabilities` descreve a decomposição dos privilégios tradicionalmente associados ao root em capacidades independentes e por *thread* [16]. Para o RunSpec, isso significa que a modelagem correta não é “usuário root ou não root”, mas sim “qual conjunto exato de capacidades é concedido nos conjuntos efetivo, permitido, *bounding*, herdável e ambiente”. O OCI já modela esse detalhe [6]. O projeto deve resistir à tentação de reduzir tudo a um booleano `privileged`. Se precisar oferecer esse atalho por compatibilidade externa, ele deve ser traduzido para uma expansão explícita no envelope de segurança, nunca tratado como semântica nativa do domínio.

O mesmo vale para `seccomp`. A documentação do Docker explica que seu perfil padrão é baseado em *allowlist* e recomenda que ele só seja alterado quando necessário [19]. A documentação do kernel deixa claro que `seccomp` filtra syscalls com base em BPF [17]. Para o seu RunSpec, isso se traduz em uma necessidade de modelar perfis `seccomp` como referências versionadas ou como documentos compiláveis, não como um campo textual livre que cada parte do sistema interpreta de um jeito. Idealmente, o contrato admite um `seccomp_profile_ref`, um `default_profile` bem definido e talvez uma forma controlada de *overrides*. Isso aumenta auditabilidade e reduz risco de combinações inconsistentes.

Em relação a AppArmor e SELinux, vale a mesma lógica. O Docker usa um perfil AppArmor padrão

chamado `docker-default` quando nenhum outro é fornecido [20]. O OCI oferece campos para `apparmorProfile`, `selinuxLabel` e `mountLabel` [6][7]. No seu desenho, a recomendação é considerar esses mecanismos como *providers* de uma mesma capacidade de domínio: impor perfis LSM e rotulagem de segurança quando disponíveis. Isso sugere o uso de *Strategy* e *Adapter* na camada nativa, com o RunSpec carregando a intenção de segurança de forma relativamente agnóstica e o *data plane* resolvendo o provedor concreto que o host suporta.

A identidade de usuário também faz parte do envelope de segurança. O Docker suporta modo `rootless`, em que `daemon` e contêineres rodam dentro de um namespace de usuário sem privilégios elevados [20], e também `userns-remap`, que mapeia o `root` do contêiner para um UID/GID não privilegiado do host por meio de `subuids` e `subgids` [21]. Para o seu projeto, a mensagem é clara: a modelagem do RunSpec deve prever desde cedo mapeamentos de usuário e grupo, mesmo que a primeira versão entregue apenas uma parte dessa capacidade. Segurança estrutural melhora muito quando o sistema não assume que “`root` no contêiner” equivale a “`root` no host”.

Por fim, `maskedPaths`, `readonlyPaths`, *read-only rootfs* e bloqueios de `sysctls` sensíveis são recursos de defesa em profundidade que pertencem claramente ao bloco `security`. O OCI os modela de forma explícita [6][7]. Em vez de tratá-los como hacks de implementação, o projeto deve reconhecê-los como parte integrante do contrato de execução.

Capítulo 8

O contexto do projeto: Python no control plane, Cython na fronteira e C++ no data plane

A divisão tecnológica proposta pelo usuário é não apenas viável, mas arquiteturalmente correta para a classe de problema em questão. Python é excelente para modelagem de domínio, API, persistência, composição semântica, validação rica, observabilidade de alto nível e evolução rápida do *control plane*. C++ é a escolha natural para a parte que exige baixo overhead, interação detalhada com o kernel, supervisão de processos, gestão fina de descritores, tratamento de erros próximo de syscalls, ciclo de vida de namespaces e cgroups e implementação de componentes duráveis como *shims* e supervisores. Cython, quando bem usado, é uma ponte sólida entre esses mundos porque permite declarar tipos C/C++, reduzir overhead de conversão, liberar o GIL em seções nativas e manter uma experiência de integração mais direta que IPC puro quando a arquitetura assim permitir [25][26].

O ponto decisivo aqui é não usar Cython como lugar de lógica de domínio. A tentação em sistemas híbridos é deixar a cola se tornar meio domínio, meio serialização, meio regra operacional. Isso costuma gerar a pior camada do sistema: difícil de testar, difícil de evoluir e sem responsabilidade clara. O melhor uso de Cython é como fronteira de transporte tipado e de tradução de ownership. O domínio vive em Python. A execução vive em C++. Cython apenas converte estruturas, chama operações coarse-grained, traduz exceções e coordena o GIL.

Também é importante não concluir, a partir do uso de Cython, que toda a relação entre Python e C++ deve ser in-process. Para certas operações, isso é aceitável e até desejável. Mas o *data plane* de um runtime de contêineres também precisa ter propriedades de durabilidade e recuperação. O *containerd* resolveu isso com *shims* que sobrevivem ao processo cliente e continuam supervisionando o contêiner [12]. O seu projeto pode adotar ideia semelhante: o *control plane* em Python conversa com um gerenciador nativo que, por sua vez, cria ou delega a *shims* por contêiner ou por grupo de contêineres. Cython pode acionar esse gerenciador, mas o estado operacional de execução não deve depender da vida do processo Python.

Essa distinção é crucial do ponto de vista de confiabilidade. Se o *control plane* cai, o contêiner não deveria desaparecer por esse motivo. Se o usuário pedir logs, status ou kill depois de uma reinicialização do daemon Python, o sistema precisa ser capaz de reconciliar com o estado observado pelo supervisor nativo. Portanto, Cython é a cola entre linguagens, não o mecanismo de acoplamento entre disponibilidade do plano de controle e sobrevivência do processo isolado.

Capítulo 9

Proposta de System Design

O desenho de sistema mais consistente para essa feature é uma arquitetura em camadas com separação explícita entre intenção, admissão e execução. Na camada mais alta, um serviço de API em Python recebe requisições externas. Essas requisições são transformadas em um modelo de intenção, que pode ser permissivo, ergonômico e até compatível com semânticas de mercado. Em seguida, um serviço de compilação de especificação combina essa intenção com metadados de imagem, perfis da plataforma, políticas de segurança, resolução de armazenamento e padrões de recursos. O resultado é um RunSpec interno canônico. Esse RunSpec é persistido como artefato de domínio, com versão, identidade e hash. Só então ele é enviado ao *data plane*.

No lado nativo, o *data plane* recebe esse RunSpec e o transforma em um plano de execução. Isso pode incluir construção do *bundle* OCI, preparação de *rootfs*, resolução final de *mounts*, criação de *cgroup*, seleção e compilação de perfis *seccomp*, aplicação de AppArmor ou SELinux, criação de namespaces, *pivot_root* ou mecanismo equivalente, revalidação de *cwd*, abertura de *stdio*, *fork/exec*, acompanhamento do processo e emissão de eventos. Parte dessas ações pode ser desempenhada por um supervisor central; outra parte pode viver em *shims* especializados, à maneira do runtime v2 do containerd [12].

A relação entre as camadas fica mais clara quando vista como fluxo de artefatos, e não apenas de chamadas. O usuário produz uma intenção. O *control plane* produz um RunSpec. O *data plane* produz um *ExecutionPlan* e um estado observado. O metastore persiste intenção, RunSpec, estado desejado, estado observado, eventos e talvez um artefato OCI renderizado para inspeção. Essa modelagem baseada em artefatos melhora muito a capacidade de depurar o sistema e reconciliar falhas.

A tabela a seguir resume a distribuição recomendada de responsabilidades.

Camada	Linguagem	Responsabilidade principal	Observação de desenho
API, domínio, metadados e políticas	Python	Receber intenção, validar, compilar RunSpec, persistir estado desejado, reconciliar estado observado	Deve ser a fonte de verdade do domínio e da admissão

Camada	Linguagem	Responsabilidade principal	Observação de desenho
Fronteira tipada	Cython	Converter modelos Python para estruturas nativas, traduzir erros, liberar GIL em chamadas longas	Não deve concentrar lógica de domínio
Gerente de runtime e supervisão	C++	Validar invariantes críticos, montar plano de execução, supervisionar processos, reter estado operacional	Deve sobreviver a falhas do processo Python
Aplicação ao kernel	C++	Namespaces, cgroups, mounts, security, exec, sinais, FDs	Código defensivo e orientado a RAI
Persistência operacional	Python + C++	Python guarda estado desejado e eventos; C++ mantém estado volátil local e checkpoints mínimos	A reconciliação deve ser explícita

O desenho também se beneficia de uma distinção entre estado desejado e estado observado. O RunSpec pertence ao lado desejado. O estado observado inclui PID do processo inicial, status, código de saída, caminhos efetivos de *root fs*, *mounts* materializados, *cgroup path*, perfis aplicados e outras evidências de execução. O Python deve reconciliar esses dois mundos. O C++ não deveria ser a fonte de verdade do domínio; ele deveria ser a fonte de verdade operacional do que está rodando agora.

9.1 Fluxo sugerido de criação e início de um contêiner

O fluxo sugerido pode ser descrito de forma simples. Primeiro, a API recebe a requisição de criação. Segundo, um compilador em Python busca a configuração da imagem, aplica *defaults* e políticas, valida combinações e produz um RunSpec canônico. Terceiro, o sistema persiste esse RunSpec e registra o estado desejado como “criado, ainda não iniciado”. Quarto, a ponte Cython transmite o contrato ao gerenciador nativo em uma chamada *coarse-grained*, preferencialmente baseada em esquema serializado estável. Quinto, o C++ materializa o plano de execução, prepara *root fs*, cria ou seleciona supervisor e aplica isolamento. Sexto, após validações finais, o processo é iniciado. Sétimo, o supervisor emite eventos como “iniciado”, “processo em execução”, “saiu” ou “falhou”. O Python consome esses eventos e atualiza o metastore.

Um detalhe importante nesse fluxo é a ordem das validações. A validação sintática e semântica geral deve ocorrer em Python, porque esse é o lugar do domínio e da experiência de API. Mas certas validações críticas, como garantia final de *cwd*, caminhos de *mount*, coerência de *root fs* e perfil *seccomp* compilado, devem acontecer novamente no lado nativo. A premissa correta é que a fronteira com o kernel exige validação defensiva, independentemente da boa disciplina do *control plane*.

Capítulo 10

Proposta de Low Level Design

No nível de LLD, a recomendação é modelar o RunSpec como estrutura fortemente tipada, serializável e versionada. Ele deve conter um cabeçalho com `kind`, `version`, `id`, `image_ref` ou `rootfs_ref`, `annotations` e talvez um hash do conteúdo normalizado. A seguir, deve trazer os blocos `process`, `hostname`, `mounts`, `resources` e `security`. O objeto precisa ser estável o bastante para ser armazenado em banco, emitido em eventos e utilizado como entrada única da criação do contêiner.

Uma representação conceitual plausível é a seguinte:

```
{
  "kind": "RunSpec",
  "version": "v1alpha1",
  "id": "ctr_01J...",
  "imageRef": "registry.example/app:1.2.3",
  "rootfsRef": "snapshot:sha256:...",
  "process": {
    "argv": ["/usr/local/bin/app", "serve"],
    "env": [
      {"name": "PATH", "value": "/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"},
      {"name": "APP_ENV", "value": "prod"}
    ],
    "cwd": "/app",
    "user": {
      "uid": 1000,
      "gid": 1000,
      "additionalGids": [1001]
    },
    "terminal": false
  },
  "hostname": "app-prod-01",
  "mounts": [
    {
      "kind": "bind",
      "sourceRef": "host:/srv/app/config",
      "destination": "/etc/app",
```

```

    "readOnly": true,
    "propagation": "rprivate"
  },
  {
    "kind": "tmpfs",
    "destination": "/run",
    "options": {
      "size": "64m",
      "mode": "0755"
    }
  }
],
"resources": {
  "memory": {"limitBytes": 536870912},
  "cpu": {"quotaMicros": 50000, "periodMicros": 100000},
  "pids": {"limit": 256}
},
"security": {
  "namespaces": {
    "pid": "private",
    "mount": "private",
    "ipc": "private",
    "uts": "private",
    "network": "private",
    "user": "private"
  },
  "noNewPrivileges": true,
  "capabilities": {
    "bounding": ["CAP_CHOWN", "CAP_DAC_OVERRIDE"],
    "effective": ["CAP_CHOWN"],
    "permitted": ["CAP_CHOWN", "CAP_DAC_OVERRIDE"]
  },
  "seccompProfileRef": "default-linux-v1",
  "apparmorProfile": "runtime-default",
  "readOnlyRootfs": true,
  "maskedPaths": ["/proc/kcore"],
  "readonlyPaths": ["/proc/sys", "/proc/sysrq-trigger"]
}
}

```

Esse JSON é apenas ilustrativo, mas ele mostra uma decisão arquitetural importante: o RunSpec deve representar intenção admitida, não necessariamente caminhos físicos finais. Por isso aparece `sourceRef` em vez de um caminho resolvido definitivo do host. A resolução concreta pode ficar no `ExecutionPlan`. Essa escolha reduz vazamento de detalhes do nó para o domínio e favorece reprodutibilidade.

10.1 Pipeline de compilação do RunSpec no control plane

O LLD do lado Python deveria organizar a admissão como um pipeline bem definido. A primeira etapa recebe o objeto de intenção e aplica validação sintática e de tipos. A segunda integra metadados da imagem, inclusive Env, Entrypoint, Cmd, WorkingDir, usuário padrão e quaisquer anotações úteis [3][5]. A terceira aplica políticas da plataforma, como perfis de segurança, limites padrão, proibição de certos *mounts* e restrições de capacidades. A quarta resolve precedências e produz campos explícitos: *argv* final, *cwd* final, *env* final, recursos finais, envelope de segurança e *mounts* normalizados. A quinta verifica invariantes cruzados, como coerência entre *hostname* e namespace UTS, entre *cwd* e topologia de *mounts*, entre *rootless* e capacidades, entre *readOnlyRootfs* e necessidades de escrita. A sexta congela o objeto, calcula seu hash e persiste o resultado.

Esse pipeline deve ser idempotente. Dada a mesma intenção, a mesma imagem e as mesmas políticas, o mesmo RunSpec deve ser produzido byte a byte. Essa propriedade é extremamente valiosa para testes, auditoria, *cache* e explicabilidade. Ela também facilita uma técnica útil: armazenar não apenas o RunSpec, mas seu hash como identidade de conteúdo. Isso permite diferenciar “esta execução pediu o mesmo contrato” de “esta execução é outra instância operacional do mesmo contrato”.

10.2 Fronteira Cython

A fronteira em Cython deve ser desenhada para maximizar estabilidade e minimizar chatice de chamadas. Em vez de expor dezenas de métodos granulares, o mais seguro é trabalhar com operações de alto nível, como `create_container(spec_blob)`, `start_container(id)`, `kill_container(id, signal)`, `delete_container(id)` e `exec_process(id, exec_spec_blob)`. O ideal é que `spec_blob` seja um formato serializado estável, porque isso reduz o atrito entre mudanças internas de Python e C++. Ainda que a implementação inicial use JSON, um formato binário versionado tende a ser melhor a longo prazo.

Do ponto de vista de Cython puro, a recomendação é declarar as classes e estruturas C++ em arquivos `.pxd`, importar essas declarações em `.pyx`, converter explicitamente tipos Python para vetores e strings nativos e executar a chamada principal dentro de `with nogil` sempre que a operação não manipular objetos Python [25][26]. Exceções C++ devem ser capturadas e convertidas em exceções de domínio do Python, preservando código de erro, mensagem, categoria e, quando aplicável, metadados de falha como qual campo do RunSpec causou rejeição. A fronteira deve ser extremamente explícita sobre ownership de memória e duração de buffers.

Também convém evitar passar callbacks Python para o runtime nativo em contexto de longa duração. Eventos de ciclo de vida deveriam ser entregues por canal próprio, como fila, socket Unix ou stream IPC, para que o C++ não dependa do GIL e não retenha referências Python em contextos que sobrevivem ao *request*. Isso torna a ponte mais robusta e facilita reinicialização do *control plane*.

10.3 Núcleo nativo em C++

No lado C++, o desenho recomendado é uma composição de componentes pequenos, especializados e orientados a RAIL. Um `RuntimeManager` recebe comandos de alto nível. Um `SpecValidator` revalida invariantes críticos. Um `BundleRenderer` ou `OCIAdapter` traduz o RunSpec para um plano nativo ou bundle OCI. Um `MountPlanner` monta a topologia de *rootfs* e *mounts*. Um `NamespaceManager` aplica e junta namespaces. Um `CgroupManager` programa *cgroup* v2. Um `SecurityApplier` resolve *seccomp*, capacida-

des, AppArmor e SELinux. Um ProcessLauncher cuida de fork/exec, descritores, sinais e reaping. Um ShimSupervisor, se adotado, mantém a vida do contêiner desacoplada da vida do daemon de controle.

O padrão de implementação a evitar aqui é o “god object do runtime” que recebe o RunSpec inteiro e executa tudo em uma função gigantesca. O domínio do problema pede explicitamente decomposição e ordem de aplicação. Para contêineres, a sequência de passos importa: preparar sistema de arquivos, isolar namespace de montagem, aplicar mounts, ajustar root, configurar UTS, PID, IPC, rede e usuário, aplicar cgroups e segurança, e só então executar o processo. Quando essa sequência fica escondida dentro de código monolítico, tanto a auditabilidade quanto a capacidade de testar falhas intermediárias despencam.

10.4 Estado operacional e supervisão

Um aspecto que merece tratamento explícito no LLD é a persistência operacional mínima do lado nativo. O containerd usa *shims* para preservar supervisão e desacoplar o ciclo de vida do contêiner do processo cliente [12]. Um projeto novo não precisa copiar isso literalmente, mas deveria absorver a lição. O contêiner não pode depender da sobrevivência do processo Python que o criou. Portanto, ou o *data plane* nativo roda como serviço próprio e durável, ou ele cria supervisores locais duráveis por instância. Em ambos os casos, deve existir um mecanismo claro de descoberta e reconciliação para que o *control plane* em Python recupere o estado em reinicializações.

Capítulo 11

Domain-Driven Design aplicado ao RunSpec

O RunSpec é um caso muito bom para aplicação de DDD porque ele vive no cruzamento entre linguagem de negócio da plataforma e mecanismos técnicos do sistema operacional. Quando um projeto não explicita essa fronteira, o jargão técnico do kernel domina tudo e o domínio de produto desaparece. Quando exagera no outro sentido, o domínio se torna abstrato demais e perde contato com as primitivas reais. O objetivo do DDD aqui é criar uma linguagem ubíqua que traduza corretamente ambos os mundos.

A recomendação é separar pelo menos os seguintes *bounded contexts*: catálogo de imagens e artefatos; admissão e compilação de especificações; execução e ciclo de vida; governança de recursos; políticas de segurança; armazenamento e montagens; observabilidade e eventos. Networking pode ser outro *bounded context* separado, e, na prática, é recomendável que fique fora do RunSpec v1 ou apareça apenas como referência a um perfil ou *sandbox* de rede. Essa decisão acompanha a observação histórica de que o Docker trata rede em configuração paralela e evita poluir o contrato de execução com detalhes demais de conectividade [5].

Dentro do contexto de admissão e compilação, o RunSpec pode ser tratado como um agregado ou, mais precisamente, como o núcleo imutável de um agregado de definição de contêiner. A entidade “ContainerDefinition” ou “ContainerInstance” pode possuir identidade operacional, enquanto o RunSpec funciona como *value object* central e versionado. A principal vantagem desse desenho é permitir que o agregado carregue metadados de ciclo de vida, estado desejado e referências sem que o contrato canônico perca seu caráter imutável.

Também é útil explicitar os eventos de domínio. Eventos como RunSpecCompiled, RunSpecRejected, BundleRendered, ContainerStarted, ContainerExited, SecurityProfileApplied, MountPlanResolved, ResourcePatchApplied e RuntimeReconciled tornam a arquitetura mais legível e ajudam a separar transições de estado de detalhes de implementação. Eles são especialmente valiosos em um sistema distribuído entre Python e C++, porque permitem uma narrativa auditável do que aconteceu.

Outro ponto forte de DDD é a noção de *anti-corruption layer*. A API pública, se inspirada em Docker, vai carregar semânticas e nomenclaturas que não precisam contaminar o centro do domínio. HostConfig, Privileged, Binds, WorkingDir, Cmd e outros termos conhecidos podem ser aceitos externamente, mas devem ser traduzidos para a linguagem ubíqua interna. Da mesma forma, OCI é uma linguagem de interoperabilidade extremamente útil, mas não precisa ser a linguagem do domínio. O RunSpec interno atua

como camada anticorrupção em ambas as direções.

Finalmente, a linguagem ubíqua deve distinguir com clareza os termos “solicitado”, “admitido”, “renderizado” e “aplicado”. Solicitado é o que o usuário pediu. Admitido é o RunSpec. Renderizado é o plano OCI ou nativo. Aplicado é o que o supervisor observou no host. Esses quatro termos, usados consistentemente em logs, eventos e documentação, evitam enorme quantidade de ambiguidade operacional.

Capítulo 12

Padrões de Design (GoF) relevantes

O desenho do RunSpec e de seu ecossistema se beneficia de vários padrões clássicos, desde que aplicados com propósito e não por ornamentação. O primeiro deles é o *Builder*. O compilador do RunSpec, em Python, essencialmente constrói um contrato a partir de múltiplas fontes: intenção externa, imagem, políticas e *defaults* da plataforma. Um *Builder* explícito ajuda a separar etapas de composição, validação e congelamento, e evita construtores gigantescos de objetos com dezenas de parâmetros.

O segundo padrão é *Strategy*. Perfis de segurança, resolução de *mounts*, seleção de provedor LSM, tradução de recursos para cgroup v2 e eventualmente suporte a múltiplos *runtimes* são variações de comportamento que não devem virar cascatas de *if* espalhadas. A estratégia permite escolher, por exemplo, entre um *SeccompStrategy*, um *AppArmorStrategy*, um *SELinuxStrategy*, um *RootlessUserStrategy* e um *RootfulUserStrategy*, sempre com contrato bem definido.

O terceiro padrão é *Adapter*. Ele aparece em pelo menos três lugares. Primeiro, no lado superior, ao adaptar uma API estilo Docker para a linguagem do domínio. Segundo, no lado inferior, ao adaptar o RunSpec para OCI ou para um plano nativo do runtime. Terceiro, no próprio Cython, ao adaptar estruturas Python para objetos C++ e vice-versa. Esse padrão é central porque sua arquitetura vive exatamente do encontro entre contratos diferentes.

O quarto padrão é *Facade*. O *control plane* não deveria conhecer a complexidade interna do *data plane*. Uma fachada nativa simples, exposta via Cython ou IPC, reduz acoplamento e deixa a camada de aplicação em Python mais legível. Em vez de lidar com múltiplos gerenciadores especializados, o Python enxerga algo como *RuntimeFacade* com poucas operações semânticas.

O quinto padrão é *State*. Contêineres percorrem estados operacionais claros: criado, iniciado, em execução, pausado se houver suporte, parado, falho, removido. Modelar isso explicitamente evita transições ilegais e ajuda na reconciliação entre estado desejado e observado. Esse padrão tem especial valor quando combinado com eventos de domínio.

O sexto padrão é *Command*. Operações como criar, iniciar, matar, deletar, executar processo extra e aplicar *patch* de recursos são comandos naturais. Tratar essas operações como objetos ou mensagens explícitas ajuda na auditoria, idempotência e repetição controlada após falhas.

O sétimo padrão é *Template Method* ou, dependendo da implementação, uma forma equivalente de pipeline explícito. A criação do contêiner segue uma sequência obrigatória de passos. Formalizar essa sequência em uma classe base ou em um fluxo estruturado evita desvios indevidos na ordem de aplicação. Já o padrão *Observer* ou uma variante de *publish/subscribe* é útil para eventos de ciclo de vida, especialmente na

comunicação do *data plane* para o *control plane*.

Nenhum desses padrões substitui o desenho de domínio, mas todos ajudam a torná-lo mais estável. O erro é aplicar padrões onde ainda falta semântica. O acerto é usá-los para cristalizar decisões de responsabilidade depois que a semântica estiver clara.

Capítulo 13

Requisitos funcionais sugeridos

Os requisitos funcionais de uma implementação séria do RunSpec podem ser organizados como contrato observável de plataforma. A tabela a seguir resume um conjunto recomendado para garantir que todos os conceitos da feature sejam realmente cobertos.

ID	Requisito funcional	Critério de aceitação resumido
FR-01	O sistema deve aceitar uma intenção de execução e compilá-la em um único RunSpec canônico	A mesma intenção, com os mesmos insumos, gera o mesmo RunSpec
FR-02	O RunSpec deve representar explicitamente <code>process</code> , <code>env</code> , <code>cwd</code> , <code>hostname</code> , <code>mounts</code> , <code>resources</code> e <code>security</code>	Nenhum desses conceitos depende de inferência implícita no runtime
FR-03	O sistema deve integrar metadados da imagem durante a compilação do RunSpec	<code>Entrypoint</code> , <code>Cmd</code> , <code>Env</code> e <code>WorkingDir</code> da imagem influenciam o resultado de forma determinística
FR-04	O sistema deve validar invariantes cruzados antes da admissão	Combinações incoerentes, como <code>hostname</code> customizado com UTS compartilhado, são rejeitadas
FR-05	O RunSpec admitido deve ser imutável e versionado	Alterações operacionais posteriores não modificam o contrato original
FR-06	O sistema deve traduzir RunSpec para um plano de execução nativo e, idealmente, para OCI	O plano final é inspecionável e reproduzível
FR-07	O data plane deve revalidar invariantes críticos antes do exec	Campos como <code>cwd</code> , <code>mounts</code> e segurança são rechecados no contexto real do host
FR-08	O sistema deve materializar e aplicar isolamento de processos, <code>mounts</code> e recursos	Namespaces, <code>cgroups</code> e <code>mounts</code> tornam-se observáveis no estado do contêiner

ID	Requisito funcional	Critério de aceitação resumido
FR-09	O sistema deve emitir eventos de ciclo de vida e manter estado observado reconciliável	O control plane consegue reconstruir o estado após reinicialização
FR-10	O sistema deve suportar processo inicial e processos extras por exec com contrato próprio	ExecSpec ou equivalente herda contexto do contêiner e define processo de forma explícita
FR-11	O sistema deve permitir política de segurança padrão e sobreposições controladas	Perfis default podem ser aplicados sem ambiguidade
FR-12	O sistema deve persistir o RunSpec e expor inspeção do contrato admitido e do plano aplicado	Operadores conseguem comparar solicitado, admitido e aplicado

Esses requisitos funcionais devem ser lidos em conjunto com o princípio de determinismo. A implementação não deve apenas “aceitar parâmetros”; ela precisa produzir um contrato estável, reproduzível e auditável. Um teste funcional bem desenhado para essa feature não é meramente “o contêiner subiu”, mas “o contêiner subiu exatamente com o contrato que foi admitido”.

Capítulo 14

Requisitos não funcionais sugeridos

A utilidade real do RunSpec depende ainda mais dos requisitos não funcionais do que dos funcionais. A tabela a seguir resume o conjunto mínimo recomendável.

ID	Requisito não funcional	Implicação arquitetural
NFR-01	Determinismo	O compilador de RunSpec deve produzir saída estável para os mesmos insumos
NFR-02	Segurança por padrão	Menor privilégio, rootfs preferencialmente somente leitura, capacidades mínimas, seccomp default e revalidação nativa
NFR-03	Auditabilidade	O sistema deve preservar solicitado, admitido, renderizado e aplicado com identidade própria
NFR-04	Evolutividade de esquema	O RunSpec deve ser versionado e compatível com migração gradual
NFR-05	Confiabilidade operacional	O contêiner não pode depender da vida do processo Python que o criou
NFR-06	Performance na fronteira de linguagens	Cython deve minimizar cópias e liberar o GIL em chamadas longas
NFR-07	Portabilidade controlada	O domínio pode ser agnóstico, mas a primeira implementação deve ser explicitamente Linux-first

ID	Requisito não funcional	Implicação arquitetural
NFR-08	Observabilidade	Eventos, logs estruturados, métricas e estado consultável devem existir em todas as transições críticas
NFR-09	Testabilidade	Deve haver testes unitários, de integração, de compatibilidade OCI e de regressão de segurança
NFR-10	Recuperação e reconciliação	Após falha ou reinício, o control plane deve reencontrar processos vivos e estados persistidos

Há dois requisitos não funcionais que merecem destaque especial. O primeiro é a segurança da fronteira Python/C++. O *control plane* não deve ser tratado como intrinsecamente confiável pelo *data plane*. O segundo é a estabilidade do contrato de dados. Se toda pequena mudança interna exigir sincronizar múltiplas estruturas frágeis entre linguagens, a produtividade que Python traria no controle será perdida em custo de manutenção na fronteira.

Capítulo 15

A decisão mais importante de design: fazer do RunSpec um contrato de domínio, não um espelho do kernel

Um risco comum ao desenhar runtimes é deixar o modelo interno se tornar apenas um espelho das estruturas do kernel ou do OCI. Isso parece prático no começo, mas tem um custo alto. O domínio deixa de expressar intenções de produto e passa a expor diretamente detalhes de aplicação. Aí surgem APIs internas difíceis de evoluir, porque qualquer mudança de semântica do produto parece exigir mudança em estruturas muito baixas.

No seu caso, o melhor caminho é fazer o contrário. O RunSpec deve ser um contrato de domínio rigoroso, compilado posteriormente para o modelo do kernel. Isso não significa esconder a realidade técnica. Significa reconhecer que, por exemplo, um perfil de segurança, um perfil de recursos ou um *mount* gerenciado são conceitos de domínio que podem ser renderizados para vários detalhes de OCI ou de syscalls. Ao manter essa separação, você preserva clareza e evita transformar o centro do sistema em um agregado de detalhes operacionais difíceis de explicar ao usuário e aos mantenedores.

A maturidade desse desenho aparece quando o sistema consegue responder de forma precisa a perguntas como: “qual foi a intenção?”, “qual foi o contrato admitido?”, “como ele foi renderizado para o host?” e “o que o host realmente aplicou?”. Se o RunSpec for apenas um espelho do `config.json`, essa conversa fica curta demais para o domínio. Se for apenas um objeto de UX, ela fica vaga demais para o runtime. O valor está exatamente no meio.

Capítulo 16

Plano de implementação incremental

Uma implementação prudente deveria evoluir em fases, mas sem perder a visão arquitetural final. A primeira fase deveria buscar uma fatia vertical mínima e segura. Nela, o sistema aceitaria intenção de execução, produziria um RunSpec canônico, persistiria esse contrato, traduziria para OCI, criaria `rootfs`, aplicaria namespaces essenciais, *mounts* básicos, `cgroup v2` para memória, CPU e PIDs, um perfil `seccomp default`, capacidades mínimas e iniciaria o processo. Nessa fase, `rootless`, `AppArmor/SELinux` avançados, dispositivos complexos e rede sofisticada podem ficar reduzidos ou mesmo fora do escopo, desde que o contrato já reserve lugar para eles.

A segunda fase deveria consolidar a robustez operacional. Isso inclui supervisor durável ou *shim*, reconciliação após reinicialização do *control plane*, eventos persistidos, inspeção completa de solicitado/admitido/aplicado, suporte a `ExecSpec`, atualização de recursos mutáveis e endurecimento de segurança para `cwd`, *mounts*, `rootfs` somente leitura e políticas de variáveis de ambiente. É também a fase em que testes de regressão de segurança precisam ganhar força.

A terceira fase pode ampliar governança e compatibilidade. Entram aqui `rootless` completo, `userns - remap`, provedores LSM mais sofisticados, tratamento de dispositivos, perfis de segurança versionados, separação mais rica de perfis de recursos e talvez uma camada de compatibilidade externa mais fiel ao Docker. Note que a ordem é deliberada: primeiro contrato e execução mínima segura; depois durabilidade e reconciliação; por fim amplitude funcional e compatibilidade de mercado.

A tabela a seguir resume esse plano.

Fase	Objetivo	Entregáveis principais
Fase 1	Contrato canônico e execução mínima segura	RunSpec v1, compilador em Python, tradução OCI, criação/início/parada/remoção, <code>cgroup v2</code> básico, <i>mounts</i> básicos, perfil <code>seccomp default</code>
Fase 2	Robustez operacional	Supervisor durável, reconciliação, eventos, inspeção completa, <code>ExecSpec</code> , revalidação forte de <code>cwd</code> , políticas de <code>mount</code> e <code>env</code>

Fase	Objetivo	Entregáveis principais
Fase 3	Ampliação de segurança e compatibilidade	Rootless, user namespaces completos, AppArmor/SELinux mais ricos, dispositivos, perfis avançados, compatibilidade externa expandida

Capítulo 17

Estratégia de testes e validação

Uma feature dessa natureza só se consolida com estratégia de testes que respeite a diversidade de falhas possíveis. Testes unitários em Python devem cobrir composição de `Entrypoint` e `Cmd`, precedência de `env`, normalização de `cwd`, regras de `hostname`, conflitos de `mounts`, perfis de recursos e compilação do envelope de segurança. Testes *golden* devem verificar que um conjunto de intenções produz exatamente o `RunSpec` esperado e, a partir dele, exatamente o OCI esperado. Esse tipo de teste é muito poderoso porque revela regressões de semântica com rapidez.

No lado C++, testes de integração precisam exercitar syscalls reais em ambiente Linux apropriado. Isso inclui criação de namespaces, montagem de `rootfs`, aplicação de `cgroup v2`, validação de `seccomp` e capacidades, reap de processos e cenários de falha no meio do pipeline. Para segurança, convém ter casos específicos de tentativa de *breakout* por `cwd`, `mounts` maliciosos, links simbólicos, sobreposição de caminhos sensíveis, variáveis de ambiente proibidas, *device nodes* indevidos e uso inesperado de capacidades. A vulnerabilidade recente do `runc` é argumento suficiente para tratar `cwd` como alvo de testes dedicados [23].

Também vale a pena aproximar o projeto do ecossistema OCI desde cedo. Se o sistema traduz `RunSpec` para OCI, é natural usar isso para executar baterias de compatibilidade e verificar se o que foi gerado respeita o espírito da especificação [6][7][8]. O teste mais valioso para esse tipo de arquitetura não é “minha API responde 200”, mas “meu contrato interno preserva semântica da ponta superior e produz ambiente correto na ponta inferior”.

Capítulo 18

Observações específicas sobre alguns campos críticos

18.1 Por que privileged não deveria ser conceito nativo do domínio

O Docker expõe Privileged como atalho poderoso e prático [4][5]. Mas, do ponto de vista de um projeto novo, ele é uma abstração ruim para o centro do domínio. “Privileged” na prática significa um pacote de relaxamentos: mais capacidades, menos restrições de dispositivos, menos isolamento de segurança e outros efeitos que variam conforme implementação. Em vez de tratar isso como conceito fundamental do RunSpec, o ideal é modelar explicitamente os componentes do envelope de segurança. Se um adaptador externo quiser aceitar privileged, ele deve traduzi-lo para uma expansão clara e auditável. O domínio central não ganha clareza ao manter esse atalho como semântica primária.

18.2 Por que rede deveria ficar fora do RunSpec v1

A feature solicitada não incluiu rede, e isso é um bom sinal. O contrato de execução já é suficientemente complexo sem acoplar todos os detalhes de conectividade. Historicamente, o Docker separa parte importante da configuração de rede e não a reduz apenas a um detalhe do processo [5]. Em um projeto novo, o mais prudente é tratar rede como subsistema paralelo ou referenciado, por exemplo por um `network_sandbox_ref`. Isso mantém o RunSpec focado em processo, filesystem, recursos e segurança, que já formam um núcleo coeso. Misturar detalhes de rede cedo demais tende a inflar o contrato e dificultar evolução.

18.3 Por que o data plane precisa desconfiar do control plane

Sistemas de plataforma frequentemente falham por excesso de confiança entre camadas. Em teoria, o *control plane* em Python já validou tudo. Na prática, bugs existem, versões divergem, serializações truncam dados e suposições envelhecem. O *data plane* toca kernel, descritores, `pivot_root`, namespaces e cgroups. Esse é o lugar onde a confiança deve ser mínima e a revalidação deve ser máxima. Essa não é uma crítica ao Python; é uma regra de engenharia de sistemas de isolamento. O módulo mais próximo do dano potencial precisa ser o mais defensivo.

Capítulo 19

Conclusão

O RunSpec interno proposto não é um detalhe cosmético do projeto; ele é a forma mais importante de estabilizar a arquitetura inteira. Em um Docker-like system implementado com Python no *control plane*, C++ no *data plane* e Cython como cola, o RunSpec deve funcionar como contrato único de execução, ponto de congelamento da admissão e fronteira semântica entre intenção declarativa e aplicação concreta no kernel. Sua função é tornar explícito aquilo que, em muitos sistemas, fica espalhado entre API, daemon, *runtime* e código histórico.

A análise do Docker original e do ecossistema OCI mostra que os conceitos pedidos pelo usuário — *process spec*, *env*, *cwd*, *hostname*, *mounts*, *resources* e *security* — não são acessórios. Eles constituem exatamente os blocos que definem a execução real de um contêiner [3][4][5][6][7]. O valor do projeto está em reorganizá-los de forma mais limpa do que o Docker histórico, sem perder compatibilidade conceitual com aquilo que o mercado já aprendeu. Isso significa aceitar que a API externa pode continuar amigável e até familiar, mas fazer do RunSpec um contrato interno mais rigoroso, versionado e imutável.

Do ponto de vista de System Design, isso conduz a uma arquitetura em que Python compila e governa o contrato, Cython transporta e sincroniza semânticas com baixo atrito, e C++ aplica o contrato com rigor operacional e defensivo. Do ponto de vista de LLD, isso pede estruturas tipadas, serialização estável, revalidação nativa, supervisão durável e decomposição explícita do pipeline de execução. Do ponto de vista de DDD, isso pede uma linguagem ubíqua baseada em solicitado, admitido, renderizado e aplicado, com o RunSpec como centro do agregado de definição de contêiner. Do ponto de vista de GoF, isso favorece *Builder*, *Strategy*, *Adapter*, *Facade*, *State*, *Command* e uma forma explícita de pipeline. Do ponto de vista de requisitos, isso exige determinismo, segurança por padrão, auditabilidade, observabilidade, testabilidade e evolução controlada do esquema.

Se a implementação respeitar esses princípios, o RunSpec deixará de ser apenas um “objeto com campos” e passará a ser o principal mecanismo de governança técnica do sistema. E esse é precisamente o ponto em que um Docker-like system deixa de ser um experimento de processos isolados e começa a se tornar uma plataforma de execução séria.

Capítulo 20

Referências

- [1] Docker Inc. *Docker overview*. Disponível em: <https://docs.docker.com/get-started/docker-overview/>.
- [2] Docker Inc. *OCI runtime support with Docker Engine*. Disponível em: <https://docs.docker.com/engine/daemon/alternative-runtimes/>.
- [3] Moby Project. *Config struct in api/types/container/config.go*. Disponível em: <https://github.com/moby/moby/blob/master/api/types/container/config.go>.
- [4] Moby Project. *HostConfig and Resources in api/types/container/hostconfig.go*. Disponível em: <https://github.com/moby/moby/blob/master/api/types/container/hostconfig.go>.
- [5] Docker Inc. *Engine API v1.24 — Create a container*. Disponível em: <https://docs.docker.com/reference/api/engine/version/v1.24/>.
- [6] Open Container Initiative. *Runtime Specification — config.md*. Disponível em: <https://github.com/opencontainers/runtime-spec/blob/main/config.md>.
- [7] Open Container Initiative. *Runtime Specification — config-linux.md*. Disponível em: <https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md>.
- [8] Open Container Initiative. *Runtime Specification — principles and operations*. Disponível em: <https://github.com/opencontainers/runtime-spec>.
- [9] containerd Authors. *Getting started with containerd client and OCI spec generation*. Disponível em: <https://github.com/containerd/containerd/blob/main/docs/getting-started.md>.
- [10] containerd Authors. *tasks.proto — CreateTaskRequest and ExecProcessRequest*. Disponível em: <https://github.com/containerd/containerd/blob/main/api/services/tasks/v1/tasks.proto>.
- [11] containerd Authors. *Runtime v2 and shim model*. Disponível em: <https://github.com/containerd/containerd/blob/main/core/runtime/v2/README.md>.
- [12] opencontainers/runc. *README and OCI bundle expectations*. Disponível em: <https://github.com/opencontainers/runc>.
- [13] Linux man-pages project. *namespaces(7)*. Disponível em: <https://man7.org/linux/man-pages/man7/namespaces.7.html>.
- [14] The Linux Kernel. *Control Group v2*. Disponível em: <https://docs.kernel.org/admin-guide/cgroup-v2.html>.

- [15] Linux man-pages project. *capabilities(7)*. Disponível em: <https://man7.org/linux/man-pages/man7/capabilities.7.html>.
- [16] The Linux Kernel. *Seccomp BPF*. Disponível em: https://docs.kernel.org/userspace-api/seccomp_filter.html.
- [17] The Linux Kernel. *No New Privileges Flag*. Disponível em: https://docs.kernel.org/userspace-api/no_new_privs.html.
- [18] Docker Inc. *Seccomp security profiles for Docker*. Disponível em: <https://docs.docker.com/engine/security/seccomp/>.
- [19] Docker Inc. *AppArmor security profiles for Docker*. Disponível em: <https://docs.docker.com/engine/security/apparmor/>.
- [20] Docker Inc. *Rootless mode*. Disponível em: <https://docs.docker.com/engine/security/rootless/>.
- [21] Docker Inc. *Isolate containers with a user namespace*. Disponível em: <https://docs.docker.com/engine/security/userns-remap/>.
- [22] Docker Inc. *Bind mounts*. Disponível em: <https://docs.docker.com/engine/storage/bind-mounts/>.
- [23] opencontainers/runc. *GHSA-xr7r-f8xq-vfvv / CVE-2024-21626*. Disponível em: <https://github.com/opencontainers/runc/security/advisories/GHSA-xr7r-f8xq-vfvv>.
- [24] Linux man-pages project. *pivot_root(2)*. Disponível em: https://man7.org/linux/man-pages/man2/pivot_root.2.html.
- [25] Cython Project. *Using C++ in Cython*. Disponível em: https://cython.readthedocs.io/en/latest/src/userguide/wrapping_CPlusPlus.html.
- [26] Cython Project. *Cython and the GIL*. Disponível em: <https://cython.readthedocs.io/en/latest/src/userguide/nogil.html>.
- [27] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [28] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [29] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.